

Testing

이준서



목차

- Testing 목적 및 기법
- mock을 이용한 Unit testing
 - 의존성
 - mock
 - testify mock
 - Unit testing 예제
- 테스트케이스 설정

Testing 목적 및 기법



testing 목적

➤ testing 목적

- 소프트웨어가 요구 사항을 충족하는지 개발자와 고객에게 입증하는 것
 - 검증테스트
- 소프트웨어의 동작이 잘못되었거나, 바람직하지 않거나, 사양에 부합하지 않는 상황을 발견하고 대응하는 것
 - 결함테스트

testing 기법 분류

- 테스트 접근 방식에 따른 분류
- 테스트 단계에 따른 분류

테스트 접근 방식에 따른 분류

- 블랙박스 테스트
 - 내부 코드 구조를 알지 못한 상태에서 소프트웨어 외부 동작을 테스트
- 화이트박스 테스트
 - 소프트웨어의 내부 구조와 코드를 이해한 상태에서 테스트
- 그레이박스 테스트
 - 일부 내부 구조에 대한 이해를 바탕으로 테스트 수행
 - 특정 모듈의 구현방식이나 데이터베이스 스키마

테스트 단계에 따른 분류

➤ Unit Testing

- 소프트웨어의 가장 작은 단위인 함수, 메서드, 클래스 등을 독립적으로 테스트
- 주로 화이트박스 테스트

➤ Integration Testing

- 여러 모듈이나 구성 요소가 함께 작동할 때 발생할 수 있는 문제 찾기 위한 테스트
- 주로 그레이박스 테스트

➤ System Testing

- 전체 시스템이 통합된 상태에서 요구사항을 충족하는지 확인하는 테스트
- 주로 블랙박스 테스트

mocking Unit testing

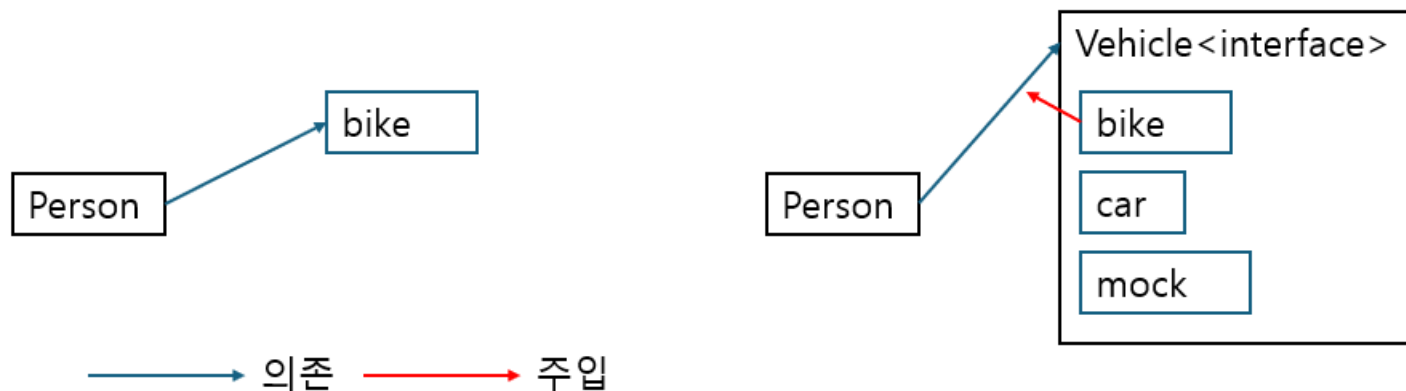


의존성

- 소프트웨어 개발에서 한 모듈이나 클래스, 함수 등이 다른 모듈, 클래스, 함수 또는 시스템에 의존하는 관계
 - 한 부분의 코드가 제대로 작동하기 위해 다른 코드나 시스템의 기능을 사용하거나 참조
- 외부 의존성
 - 데이터베이스, 파일시스템, 외부 API, 네트워크 서비스 등 코드 외부 시스템이나 서비스에 의존하는 경우

의존성 주입

- 의존성을 관리하고 코드를 더 유연하고 테스트 가능하게 만드는 디자인 패턴
 - 객체가 필요로 하는 의존성을 인터페이스 형태로 정의하고 외부에서 구체적인 구현체 주입 받음
 - Loose Coupling으로 코드 유지보수 수월
 - ✓ 의존하고 있는 객체 변경에 미치는 영향 최소화
 - 생성자 주입 방법으로 의존성 주입
 - ✓ 생성자 실행할 때 실제 객체 주입
 - 의존성 쉽게 교체할 수 있고 테스트 시에 Mock 객체로 대체 가능



<의존성 주입 예시 그림>

Mock 정의 및 목적

- Mock 정의
 - 특정 객체의 동작을 시뮬레이션하는 가짜 객체 또는 가짜 함수
- Mock 목적
 - 외부 의존성 제거
 - 특정 시나리오 시뮬레이션
 - 동작 검증
 - 테스트 속도 향상

Unit test에서의 mock 사용 이유

- 외부 의존성 제거
 - 독립적인 실행으로 신뢰성 향상
- 특정 시나리오 시뮬레이션
 - 예외, 간헐적 에러를 반환하도록 설정하여 실제 환경에서 재현하기 어려운 상황 또한 테스트
- 동작 검증
 - 코드의 예상된 동작을 확인할 수 있음
 - stubbing으로는 불가능함
 - 외부의존성을 간단한 대체 객체로 대체하는 것
- 테스트 속도 향상
 - 즉각적인 응답 반환
- 테스트 범위와 집중도 향상
 - 나머지 코드 간의 상호작용 차단하여 테스트 대상 코드만을 검증 가능함

go 언어에서 mocking unit test가 효과적인 이유

➤ 인터페이스 기반 설계

- 의존성 주입이 인터페이스를 통해 이루어짐
 - 특정 객체의 동작 추상화하고 해당 인터페이스 구현하는 mocking 객체 주입

➤ 동시성 모델

- goroutine이 수행하는 작업 시뮬레이션할 수 있음
 - 비동기 코드 테스트에 유리함
- 메서드 호출 검증
 - AssertCalled
 - ✓ 특정 인수로 메서드 호출되었는지 확인
 - AssertNumberOfCalls
 - ✓ 메서드가 정확히 몇 번 호출되었는지 검증

➤ 모듈성

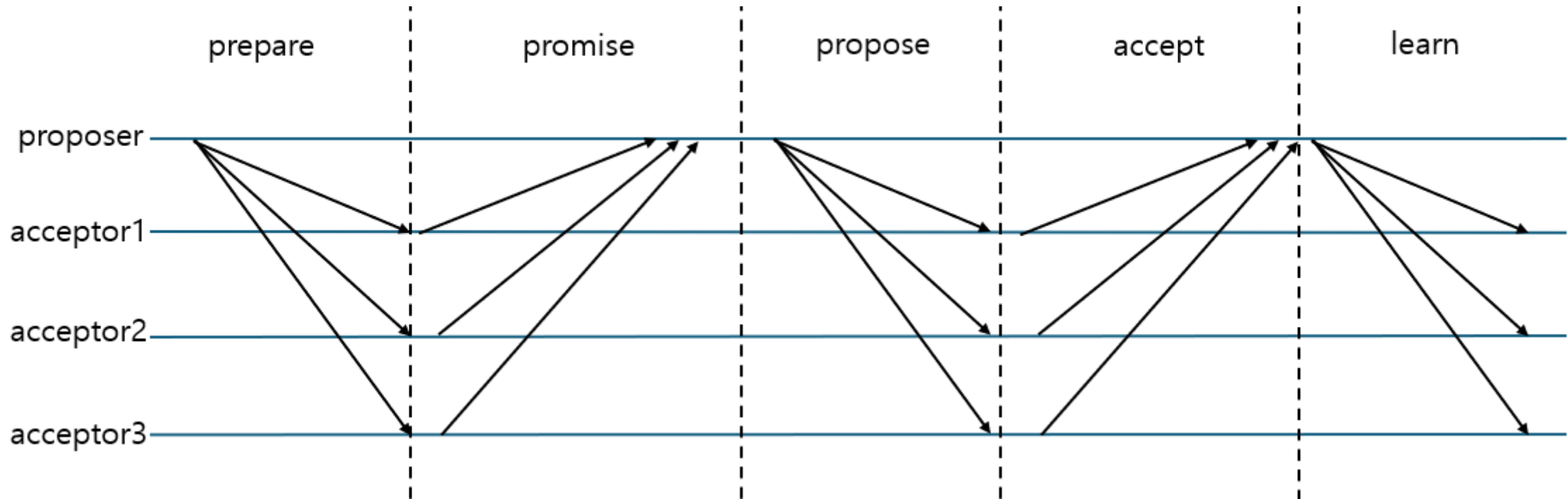
- 다른 모듈과의 의존성을 제거하고 해당 모듈의 로직에 집중한 테스트 가능

golang testing tool testify

- 기본 테스트 패키지 `testing`을 확장하여 더 많은 기능과 API 제공
 - `github.com/stretchr/testify` 모듈 가져와서 사용
- `assert` 패키지
 - 테스트에서 기대하는 결과를 검증하는데 사용
 - `assert.Equal(t, expected, actual)`
 - `assert.Nil(t, object)`
 - `assert.True(t, condition)`
- `mock` 패키지
 - 의존성을 mocking할 수 있는 기능 제공
 - `mock.On(method,arguments...)`
 - `mock.On.Return` / `mock.On.Panic` / `mock.On.Times(n)`
 - `mock.Called(arguments...)`

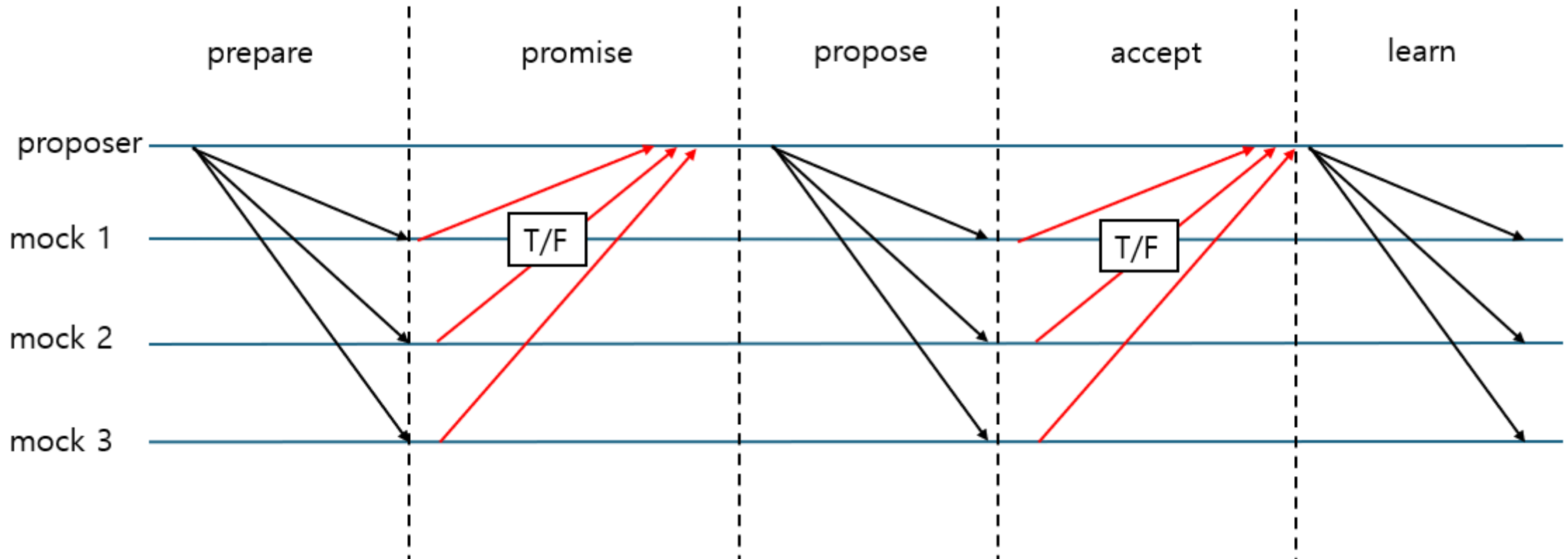
mocking unit test 예제 설명

- single paxos 알고리즘 normal case를 기준으로 구현



예제 mocking 적용 방법

- acceptor들을 mock 객체로 두고 proposer에 대한 응답을 조절하였음



테스트케이스 설정



테스트케이스 설정 주요 목표

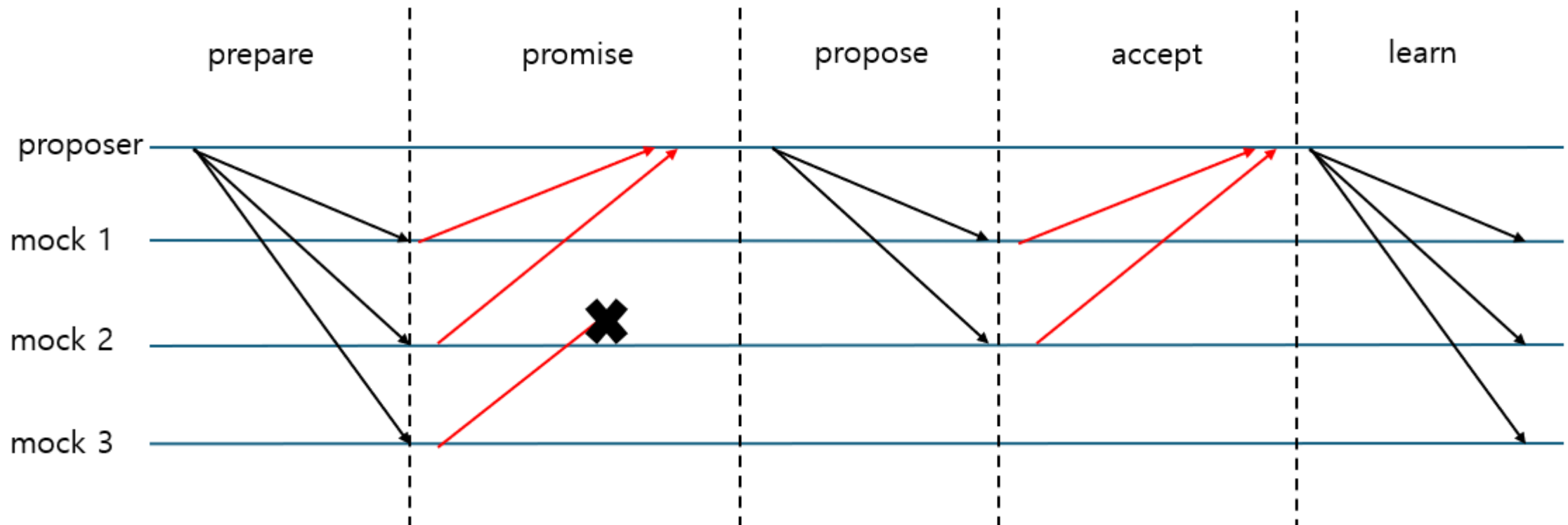
- 요구사항 검증
- 결함 발견
- 커버리지 극대화
 - 얼마나 테스트 충분한가를 나타냄
 - **branch coverage**를 중요하게 생각하고 설정할 것
 - 로직의 시나리오에 대한 테스트

예제에서 비동기상황 테스트케이스 예시

- 과반수가 아닌 acceptor의 실패
- 과반수의 acceptor 실패

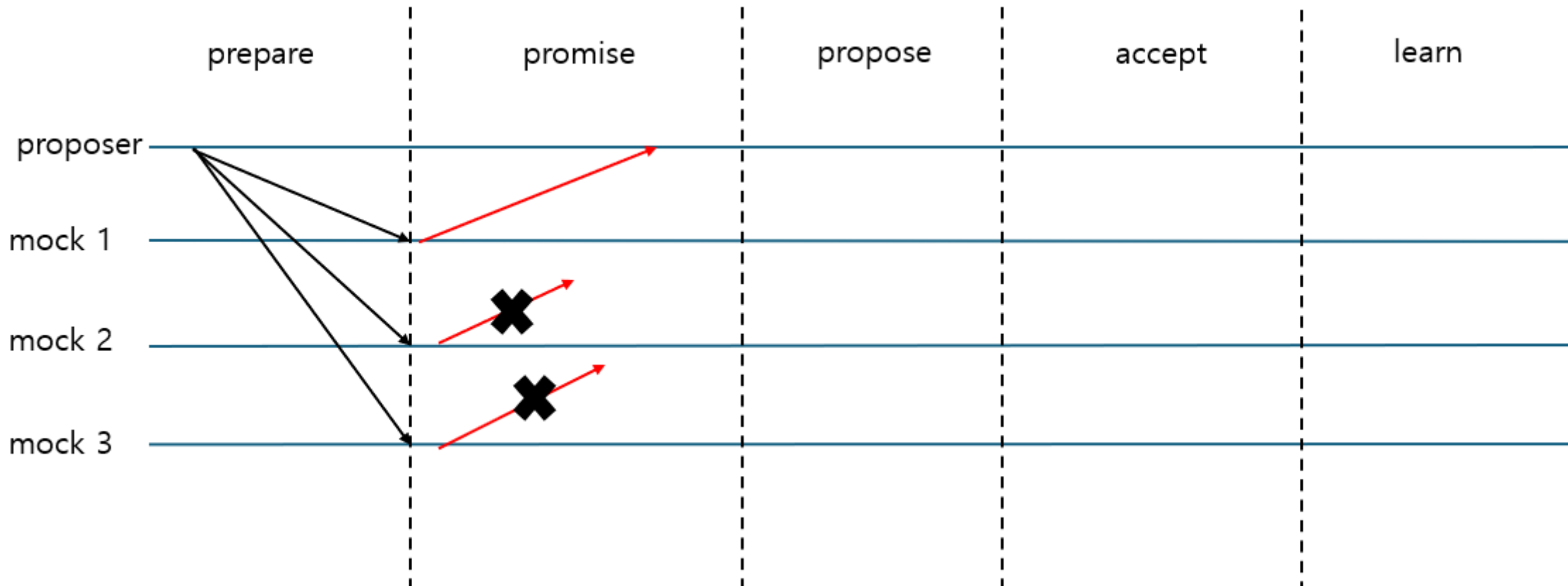
과반수가 아닌 acceptor의 실패

- mockAcceptor.On().Return(false) 의 형태로 구현 가능
- 구현하였을 때 올바르게 합의 진행하는지 test



과반수의 acceptor 실패

- 여러 객체에 대해 `mockAcceptor.On().Return(false)` 의 형태로 구현 가능
- 구현하였을 때 합의 불가능한지 test



비동기 상황에서 개발 방법

- 비동기 상황에 따른 여러 테스트케이스 만들어두고 TDD 개발
 - Test Driven Development
 - 테스트 먼저 설계하고 작성한 후 테스트 통과하도록 코드 작성하는 과정 반복
 - 각 테스트케이스는 mocking으로 구현